

간섬유화 전사인자(TF) Consensus 파이프라인

코드 워크스루 — pharmacology2254 저장소

이 문서는 이번 프로젝트에서 작성한 R 코드 12개 스크립트를 실행 순서대로 정리하고, 각 코드 블록이 무엇을 하는지, 왜 그렇게 짰는지를 설명합니다. 목표는 3개의 독립적인 공개 GEO 데이터셋(사람 대사질환, 마우스 CCl4 독성, 마우스 TAA 독성)에서 간섬유화가 진행될 때 공통으로 움직이는 상위 전사인자(TF)를 찾아, 새로운 치료 타겟 후보를 제안하는 것입니다.

이 문서의 범위

코드 자체의 흐름과 방법론 설명에 집중합니다. 문헌 조사/실험계획 등 코드가 아닌 산출물은 마지막 장에 요약만 넣었습니다 (전체 내용은 `candidate_shortlist.csv`, `ARID1A_experimental_plan.md` 참고).

연구 개요와 전체 흐름

왜 이 분석인가

약리학 연구실이 다루는 핵심 질문은 "간독성물질을 처리했을 때 세포 수준에서 어떤 변화가 일어나고, 그 변화를 완화할 수 있는 물질/타겟은 무엇인가"입니다. 유전자 하나하나를 보는 대신, 발현 변화를 지휘하는 상위 전사인자(transcription factor, TF)를 찾으면 더 적은 수의 핵심 조절자로 타겟을 좁힐 수 있습니다. 이 프로젝트는 서로 다른 종(사람/마우스), 서로 다른 병인(대사성 질환/화학독성 2종)에서 공통으로 나타나는 TF를 찾아, 병인과 무관하게 작동하는 범용 섬유화 드라이버 후보를 제안하는 것을 목표로 합니다.

전체 파이프라인 4단계

- 1단계 - DEG 분석: 각 데이터셋에서 정상(또는 대조군) 대비 섬유화가 진행된 그룹의 차이발현유전자(DEG)를 찾음 (RNA-seq은 DESeq2, microarray는 limma).
- 2단계 - TF activity 추론: DEG 결과가 아니라 정규화된 발현행렬 전체를 입력으로, decoupleR 패키지와 CollecTRI라는 TF-표적유전자 데이터베이스를 이용해 "어떤 TF가 이 발현 패턴을 만들었는가"를 샘플별로 점수화함.
- 3단계 - 데이터셋 내부 통계: 진행단계(또는 시점) 그룹 간 TF activity 차이를 ANOVA로 검정하고 BH(Benjamini-Hochberg)로 다중검정 보정.
- 4단계 - 데이터셋 간 메타분석: 3개 데이터셋의 TF별 p-value를 Fisher's method로 결합하고, "3개 모두에서 독립적으로 유의 + 방향까지 일치"라는 엄격한 기준으로 최종 후보를 추림.

사용한 3개 데이터셋

데이터셋	종/조직	축	설계	플랫폼
GSE135251	사람, 간 생검	대사성 만성질환	Normal→Steatosis→Early Fibrosis→Advanced Fibrosis (n=216)	RNA-seq
GSE222576	마우스, 간	화학독성 #1 (CCl4)	Control, 2/4/6/8/10주 (n=3/군)	Affymetrix microarray
GSE74605	마우스, 간	화학독성 #2 (TAA)	Naive, 1/4/6주 (n=3-4/군)	Illumina microarray

저장소 폴더 구조

```
pharmacology2254/  
  shared/R/  
    pub_theme.R          # 논문용 ggplot 테마/팔레트/저장 규격  
    tf_activity.R         # TF activity 추론 공용 함수 (모든 데이터셋이 재사용)  
  GSE135251_human_MASLD/  01_download_qc.R  02_deg_deseq2.R  03_tf_activity.R  
  GSE222576_mouse_CCl4/   01_download_qc.R  02_deg_limma.R   03_tf_activity.R  
  GSE74605_mouse_TAA/     01_download_qc.R  02_deg_limma.R   03_tf_activity.R  
  cross_dataset_metaanalysis/  
    01_combine_tf_scores.R # 3개 데이터셋 결과를 합치는 최종 분석  
  candidate_shortlist.csv  # 문헌/druggability triage (코드 아님)  
  ARID1A_experimental_plan.md # 실험 검증 계획 (코드 아님)
```

1. 공용 모듈: shared/R/

세 데이터셋의 플랫폼(RNA-seq vs microarray)과 종(사람 vs 마우스)이 다르지만, "DEG를 낸 뒤 정규화된 발현행렬로 TF activity를 계산한다"는 절차 자체는 동일합니다. 이 반복되는 로직을 함수로 뽑아 shared/R/에 두고 모든 스크립트가 source()해서 재사용합니다 — 코드 중복을 막고, 세 데이터셋이 정확히 같은 통계 방법을 쓴다는 걸 보장합니다.

1.1 pub_theme.R — 논문용 시각화 규격

shared/R/pub_theme.R

모든 그림(volcano plot, PCA, heatmap)이 같은 폰트/색상/저장 포맷을 쓰도록 고정하는 파일입니다. 그림마다 스타일이 제각각이면 나중에 논문 Figure로 모을 때 손이 많이 가므로, 처음부터 하나로 통일했습니다.

```
theme_pub = function(base_size = 12) {  
  theme_bw(base_size = base_size) +  
    theme(  
      panel.grid.minor = element_blank(),  
      panel.grid.major = element_line(linewidth = 0.25, color = "grey85"),  
      ...  
    )  
}
```

ggplot2 기본 테마(theme_bw)에 논문에서 흔히 쓰는 미세 조정(그리드 단순화, 굵은 제목 등)을 얹은 함수.

```
pub_palette_progression = c(  
  "Normal" = "#0072B2", "Steatosis_Only" = "#009E73",  
  "Early_NASH_Fibrosis" = "#E69F00", "Advanced_Fibrosis" = "#D55E00"  
)
```

Okabe-Ito 색맹 친화 팔레트. 진행단계 이름에 미리 색을 고정해둬서, 어떤 그림에서든 "Normal=파랑, Advanced=주황"이 항상 같게 유지됨.

```
pub_heatmap_colors = function(n = 100) {  
  rev(colorRampPalette(RColorBrewer::brewer.pal(9, "RdBu"))(n))  
}  
ggsave_pub = function(filename, plot = ggplot2::last_plot(), width = 7, height = 5) {  
  ggplot2::ggsave(filename = filename, plot = plot, width = width, height = height,  
    units = "in", device = "pdf")  
}
```

heatmap용 파란-빨강 diverging 색상, 그리고 모든 그림을 PDF(벡터)로 같은 규격으로 저장하는 래퍼 함수.

1.2 tf_activity.R — TF activity 추론 공용 함수

shared/R/tf_activity.R

이 프로젝트의 핵심 엔진입니다. decoupleR 패키지와 CollecTRI(TF-표적유전자 네트워크 데이터베이스)를 이용해 "이 발현행렬에서 어떤 TF가 활성화되어 있는가"를 계산합니다.

실행 중 발견한 버그와 우회

설치된 OmnipathR(3.18.4) 패키지의 organism-resolution 로직에 버그가 있어서 decoupleR::get_collectri()를 그대로 호출하면 항상 에러가 났습니다 (내부적으로 종(species) 이름을 NCBI taxonomy ID로 변환하는 과정에서 join이 실패). 원인을 추적해서, OmniPath REST API를 직접 호출해 decoupleR과 완전히 동일한 네트워크 구성 규칙을 재현하는 방식으로 우회했습니다. 아래 build_collectri_direct() 함수가 그 우회 로직입니다.

CollectRI 네트워크 직접 구성

```
build_collectri_direct = function(organism = "human") {
  taxid = if (is.numeric(organism)) organism else ORGANISM_TAXID[[organism]]
  url = sprintf(
    "https://omnipathdb.org/interactions?datasets=collectri&genesymbols=1&organisms=%d&fields=
    is_stimulation,is_inhibition",
    taxid
  )
  raw = read_tsv(url, show_col_types = FALSE)
```

OmniPath REST API에 직접 TSV 형식으로 TF-표적유전자 상호작용 목록을 요청. organism을 숫자 taxonomy ID로 넘겨서 OmnipathR의 버그가 있는 이름->ID 변환 단계를 아예 거치지 않게 함.

```
cols = c("source_genesymbol", "target_genesymbol", "is_stimulation", "is_inhibition")
interactions = raw[!str_detect(raw$source, "COMPLEX"), cols]
complexes = raw[str_detect(raw$source, "COMPLEX"), cols] %>%
  mutate(source_genesymbol = case_when(
    str_detect(source_genesymbol, "JUN") | str_detect(source_genesymbol, "FOS") ~ "AP1",
    str_detect(source_genesymbol, "REL") | str_detect(source_genesymbol, "NFKB") ~ "NFKB"
  ))
```

AP-1, NF-kB처럼 여러 단백질이 모여야 작동하는 "복합체" TF는 개별 서브유닛 대신 대표 이름(AP1/NFKB)으로 합쳐줌 — decoupleR::get_collectri()의 기본 동작을 그대로 재현.

```
uniprot_pattern =
  "^[OPQ][0-9][A-Z0-9]{3}[0-9](-[0-9]+)?$|^([A-NR-Z][0-9]([A-Z][A-Z0-9]{2}[0-9]){1,2}(-[0-9]+)?$)"

rbind(interactions, complexes) %>%
  distinct(source_genesymbol, target_genesymbol, .keep_all = TRUE) %>%
  mutate(mor = case_when(is_stimulation == 1 ~ 1, is_stimulation == 0 ~ -1)) %>%
  filter(!is.na(mor), !is.na(source_genesymbol)) %>%
  filter(!str_detect(source_genesymbol, uniprot_pattern),
    !str_detect(target_genesymbol, uniprot_pattern)) %>%
  select(source = source_genesymbol, target = target_genesymbol, mor)
}
```

mor(mode of regulation)을 +1(활성화)/-1(억제)로 인코딩. 일부 마우스 유전자는 정식 이름이 없어 UniProt accession(예: A0A0R4J082)이 그대로 남는데, 이런 항목이 그림에 "TF"로 노출되면 신뢰도를 해치므로 정규식으로 걸러냄 — GSE222576 결과를 처음 봤을 때 발견하고 나중에 추가한 로직.

네트워크 캐싱과 TF activity 계산

```
get_collectri_cached = function(organism = "human", cache_dir = "shared/cache") {
  if (!dir.exists(cache_dir)) dir.create(cache_dir, recursive = TRUE)
  cache_file = file.path(cache_dir, paste0("collectri_", organism, ".rds"))
  if (file.exists(cache_file)) return(readRDS(cache_file))
  net = build_collectri_direct(organism)
  saveRDS(net, cache_file)
  net
}
```

네트워크를 매 실행마다 새로 받지 않도록 로컬에 캐싱. 사람/마우스 두 번만 받으면 이후 모든 스크립트가 재사용.

```
run_tf_activity = function(expr_matrix, organism = "human", minsize = 5, cache_dir =
  "shared/cache") {
  net = get_collectri_cached(organism, cache_dir)
  acts = decoupleR::run_ulm(
    mat = as.matrix(expr_matrix), net = net,
    .source = "source", .target = "target", .mor = "mor", minsize = minsize
  )
  acts_wide = acts %>%
    select(source, condition, score) %>%
    pivot_wider(names_from = condition, values_from = score) %>%
    column_to_rownames("source") %>% as.matrix()
  list(long = acts, wide = acts_wide, network = net)
}
```

핵심 함수. decoupleR의 univariate linear model(ulm) 방법으로 각 샘플·각 TF의 활성 점수를 계산. 입력은 "gene symbol x sample" 발현행렬 하나뿐이라, RNA-seq이든 microarray든 동일하게 호출 가능.

그룹 요약과 통계 검정

```
summarize_tf_by_group = function(acts_wide, group_vector, group_levels) {
  group_vector = factor(group_vector, levels = group_levels)
  apply(group_levels, function(g) rowMeans(acts_wide[, group_vector == g, drop = FALSE],
    na.rm = TRUE))
}
```

TF activity를 진행단계별 평균으로 묶어 heatmap 입력 행렬을 만들. group_levels로 순서를 명시적으로 고정.

```
test_tf_across_groups = function(acts_wide, group_vector, group_levels, ordered = TRUE) {
  group_vector = factor(group_vector, levels = group_levels, ordered = ordered)
  pvals = apply(acts_wide, 1, function(tf_scores) {
    fit = tryCatch(aov(tf_scores ~ group_vector), error = function(e) NULL)
    if (is.null(fit)) return(NA_real_)
    summary(fit)[[1]][["Pr(>F)"]][1]
  })
  data.frame(TF = rownames(acts_wide), p_value = pvals) %>%
    filter(!is.na(p_value)) %>% mutate(padj = p.adjust(p_value, method = "BH")) %>%
    arrange(padj)
}
```

TF마다 그룹 간 ANOVA를 돌려 p-value를 구하고 BH로 다중검정 보정. "top N을 그냥 눈으로 골랐다"가 아니라 통계적으로 정당화된 랭킹을 만들기 위한 함수.

메타분석용 p-value 결합 함수

```
fisher_combine = function(pvalues) {
  pvalues = pvalues[!is.na(pvalues) & pvalues > 0]
  stat = -2 * sum(log(pvalues))
  pchisq(stat, df = 2 * length(pvalues), lower.tail = FALSE)
}
stouffer_combine = function(pvalues, weights = NULL) {
  pvalues = pmin(pmax(pvalues, 1e-300), 1 - 1e-16)
  z = qnorm(1 - pvalues)
  if (is.null(weights)) weights = rep(1, length(pvalues))
  z_comb = sum(weights * z) / sqrt(sum(weights^2))
  1 - pnorm(z_comb)
}
```

독립적인 여러 데이터셋에서 나온 같은 TF의 p-value들을 하나로 합치는 표준 메타분석 공식. cross_dataset_metaanalysis 단계에서 사용.

2. 데이터셋 1 — GSE135251 (사람 MASLD 스펙트럼)

인체 간 생검 216개의 bulk RNA-seq (Govaere et al.). 정상부터 진행된 섬유화까지 질환 스펙트럼 전체를 포함하고 있어 대사성 만성질환 축의 기준(anchor) 데이터셋으로 사용. 표본 크기가 커서(n=216) 통계적 검정력이 가장 높은 데이터셋이기도 함.

2.1 01_download_qc.R — 다운로드와 QC

GSE135251_human_MASLD/01_download_qc.R

GEO에서 raw count를 받아 병합하고, 메타데이터를 정리해서 4단계 진행 그룹으로 나눈 뒤, 저발현 유전자를 걸러내는 전처리 스크립트.

```
existing_tar = "C:/Users/SAMSUNG/Desktop/5-1 1~2/GSE135251/GSE135251/GSE135251_RAW.tar"
local_tar = file.path(raw_dir, "GSE135251_RAW.tar")
if (!file.exists(local_tar)) {
  if (file.exists(existing_tar)) { file.copy(existing_tar, local_tar) }
  else { getGEOSuppFiles("GSE135251", baseDir = raw_dir, makeDirectory = FALSE) }
}
```

216샘플짜리 큰 raw 파일을 매번 새로 받지 않도록, 이미 로컬에 받아둔 파일이 있으면 재사용하고 없을 때만 GEO에서 다운로드.

```
group_levels = c("Normal", "Steatosis_Only", "Early_NASH_Fibrosis", "Advanced_Fibrosis")

meta_final = metadata %>%
  mutate(analysis_group = case_when(
    disease == "Control" ~ "Normal",
    group == "NAFL" ~ "Steatosis_Only",
    group %in% c("NASH_F0-F1", "NASH_F2") ~ "Early_NASH_Fibrosis",
    group %in% c("NASH_F3", "NASH_F4") ~ "Advanced_Fibrosis",
    TRUE ~ "Others"
  )) %>%
  filter(analysis_group != "Others") %>%
  mutate(analysis_group = factor(analysis_group, levels = group_levels)) %>%
  arrange(analysis_group)
```

GEO 메타데이터의 원래 세부 병기(NAFL, NASH_F0-F1 등)를 저희 분석에 쓸 4단계 진행 그룹으로 재매핑. factor의 levels 순서를 명시적으로 고정해서 이후 모든 그림/통계가 항상 이 순서를 따르게 함.

```
keep = rowSums(merged_counts >= 5) >= 10
merged_counts_filtered = merged_counts[keep, ]
message(sprintf("저발현 필터링: %d genes -> %d genes", nrow(merged_counts),
  nrow(merged_counts_filtered)))
```

DESeq2 표준 권장 방식의 저발현 필터: 최소 10개 샘플에서 count 5 이상인 유전자만 남김. 노이즈를 줄여 다중검정 보정 시 통계적 유의성을 확보하기 쉽게 함.

실행 결과

64,257개 → 21,191개 유전자로 필터링. 최종 216 samples (Normal 10 / Steatosis 51 / Early Fibrosis 87 / Advanced Fibrosis 68).

2.2 02_deg_deseq2.R — DESeq2 DEG 분석

GSE135251_human_MASLD/02_deg_deseq2.R

DESeq2로 정식 통계 모델을 적용해 차이발현유전자(DEG)를 찾는 단계.

사전에 고정한 유의기준

padj(BH) < 0.05 그리고 $|\log_2FC| > 1$ 을 분석 전에 고정하고, 결과가 안 나와도 사후에 완화하지 않는다는 원칙을 세웠습니다. 과거 다른 분석(GSE202379)에서 padj 기준으로 유의 유전자가 0개가 나오자 $p < 0.01$ 로 기준을 낮춘 적이 있었는데, 이런 리뷰어가 바로 지적할 수 있는 관행이라 이번엔 반복하지 않기로 했습니다.

```
dds = DESeqDataSetFromMatrix(countData = count_final, colData = meta_final, design = ~
  analysis_group)
dds = dds[rowSums(counts(dds)) > 0, ]
vsd = vst(dds, blind = FALSE)
saveRDS(vsd, file.path(data_dir, "vsd.rds"))
```

DESeq2 객체 생성 후 VST(분산안정화변환) 적용. VST 결과는 이후 TF activity 계산 단계에서 그대로 재사용.

```
dds$analysis_group = relevel(dds$analysis_group, ref = "Steatosis_Only")
dds = DESeq(dds)

contrasts_list = list(
  Early_vs_Steatosis = "analysis_group_Early_NASH_Fibrosis_vs_Steatosis_Only",
  Advanced_vs_Steatosis = "analysis_group_Advanced_Fibrosis_vs_Steatosis_Only"
)
```

Steatosis_Only(지방간만 있고 섬유화는 아직 없는 상태)를 기준(reference)으로 삼아, "섬유화가 어느 정도 시작된 시점부터 무엇이 달라지는가"를 보는 두 가지 대비(contrast)를 정의.

```
for (nm in names(contrasts_list)) {
  res_shrunk = lfcShrink(dds, coef = contrasts_list[[nm]], type = "apeglm")
  res_df = as.data.frame(res_shrunk) %>%
    rownames_to_column("GeneID") %>% filter(!is.na(padj)) %>% arrange(padj)

  n_sig = sum(res_df$padj < SIG_PADJ & abs(res_df$log2FoldChange) > SIG_LFC)
  message(sprintf("[%s] 유의 유전자 (padj<%.2f & |log2FC|>%.1f): %d개 / 검정된 %d개 중",
    nm, SIG_PADJ, SIG_LFC, n_sig, nrow(res_df)))
  ...
}
```

apeglm으로 log2FC shrinkage(발현량 적은 유전자의 노이즈를 줄이는 논문용 필수 후처리)를 적용한 뒤, 고정된 기준으로 유의 유전자 수를 세고 volcano plot을 저장.

실행 결과

Early vs Steatosis: 21개 유의 유전자. Advanced vs Steatosis: 528개 유의 유전자 (둘 다 사전 고정 기준 그대로, 완화 없이 나온 결과).

2.3 03_tf_activity.R — TF activity 추론

GSE135251_human_MASLD/03_tf_activity.R

VST 발현행렬을 shared/R/tf_activity.R의 함수에 넣어 TF activity를 계산하고, 진행단계별 heatmap을 그리는 단계.

```
gene_map = bitr(rownames(expr), fromType = "ENSEMBL", toType = "SYMBOL", OrgDb = org.Hs.eg.db)
gene_map = gene_map[!duplicated(gene_map$ENSEMBL), ]
expr_sym = expr[gene_map$ENSEMBL, ]
rownames(expr_sym) = gene_map$SYMBOL
expr_sym = expr_sym[!duplicated(rownames(expr_sym)), ]
```

CollectRi 네트워크는 gene symbol(예: TP53) 기준인데 count 파일은 Ensembl ID(예: ENSG00000141510) 기준이라, clusterProfiler::bitr()로 ID를 변환. 이 과정에서 매핑 안 되는 유전자 17.4%는 자연스럽게 제외됨.

```
tf_result = run_tf_activity(expr_sym, organism = "human", cache_dir = "shared/cache")
tf_group = summarize_tf_by_group(tf_result$wide, meta_final$analysis_group, group_levels)
tf_stats = test_tf_across_groups(tf_result$wide, meta_final$analysis_group, group_levels,
                                ordered = TRUE)
```

공용 함수 3개를 순서대로 호출: TF activity 계산 -> 그룹별 평균 -> 그룹간 통계 검정. ordered=TRUE로 넘겨서 "진행단계에 순서가 있다"는 걸 통계 모델에 반영.

```
if (n_sig >= 5) {
  top_tf = tf_stats %>% filter(padj < 0.05) %>% slice_head(n = 30) %>% pull(TF)
  heatmap_title = sprintf("TF activity across MASLD progression (BH padj<0.05, n=%d)",
    length(top_tf))
} else {
  message("BH padj<0.05를 통과한 TF가 5개 미만 -> nominal p-value 상위 20개를 참고용으로만 표시")
  top_tf = tf_stats %>% slice_head(n = 20) %>% pull(TF)
  heatmap_title = "TF activity (top 20 by nominal p; not BH-significant)"
}
```

여기서도 같은 원칙: 통계적으로 유의한 TF가 부족하면 "참고용"이라고 제목에 명시하고, 유의하지 않은 결과를 유의한 것처럼 포장하지 않음.

실행 결과

703개 TF 평가, 518개가 BH padj<0.05로 유의 (n=216이라 검정력이 매우 높음). 상위 TF: HOPX, ELK4, NAB2, TOX3, SMARCA4, HDAC3, ARX, NCOA6, NR2E1, YAP1 — YAP1(Hippo 경로, 간성상세포 활성화의 잘 알려진 드라이버)이 상위권에 나온 게 파이프라인이 실제 생물학을 제대로 잡아내고 있다는 신호였습니다.

3. 데이터셋 2 — GSE222576 (마우스 CCl4 시계열)

CCl4를 반복 투여해 간섬유화를 유발한 마우스 모델의 시계열 microarray (Control, 2/4/6/8/10주, n=3/시점). 화학독성 측의 첫 번째 데이터셋. GSE135251과 플랫폼(RNA-seq vs microarray)도, 종(사람 vs 마우스)도, 병인(대사질환 vs 급성 화학독성)도 전부 다르다는 게 핵심 — 여기서도 같은 TF가 나온다면 우연이 아니라는 근거가 됨.

3.1 01_download_qc.R — CEL 다운로드와 RMA 정규화

GSE222576_mouse_CCl4/01_download_qc.R

Affymetrix CEL 원자료를 받아서 RMA(Robust Multi-array Average)로 정규화하는 단계. RNA-seq과 달리 microarray는 이 단계가 완전히 다른 절차임.

```
meta = pd %>%
  rownames_to_column("gsm") %>%
  mutate(
    weeks = as.integer(str_extract(title, "(?<=for )\\d+(?= weeks)")),
    group = if_else(str_detect(title, "^Control"), "Control", paste0("CCl4_", weeks, "w")),
    group = factor(group, levels = group_levels)
  ) %>%
  select(gsm, title, group, weeks)
```

GEO 메타데이터의 title 컬럼(예: "CCl4-induced liver fibrosis for 2 weeks, biological rep1")에서 정규식으로 주차 정보를 뽑아 그룹을 만들. 이 데이터셋은 별도 characteristics 필드가 없고 title에만 정보가 있어서, 실행 전에 GEOquery로 실제 메타데이터를 먼저 조회해 정확한 문자열 패턴을 확인한 뒤 코드를 짤.

```
raw_data = read.celfiles(cel_files)
eset = oligo::rma(raw_data, target = "core")
expr = exprs(eset)
colnames(expr) = meta$gsm
```

oligo 패키지로 RMA 정규화. 이 array는 Gene ST 타입이라 target="core"로 지정해 probe 단위가 아니라 transcript-cluster(=유전자에 가까운 단위) 레벨로 요약.

실행 결과

41,345 transcript clusters x 18 samples. 그룹당 정확히 3개씩 (Control/2w/4w/6w/8w/10w).

3.2 02_deg_limma.R — limma DEG 분석

GSE222576_mouse_CCl4/02_deg_limma.R

microarray는 DESeq2가 아니라 limma를 씬 (raw count가 아니라 이미 연속값으로 정규화된 log-scale 신호이기 때문).

```
probe_map = AnnotationDbi::select(mogene20sttranscriptcluster.db,
  keys = rownames(expr),
  columns = "SYMBOL", keytype = "PROBEID")
```

이 array 전용 annotation 패키지(mogene20sttranscriptcluster.db)로 probe ID -> gene symbol 매핑.

```

design = model.matrix(~ group, data = meta)
fit = lmFit(expr, design)
fit = eBayes(fit)

for (coef in timepoint_coefs) {
  nm = str_remove(coef, "^group")
  tt = topTable(fit, coef = coef, number = Inf, adjust.method = "BH") %>%
    rownames_to_column("PROBEID") %>% left_join(probe_map, by = "PROBEID") %>%
    filter(!is.na(SYMBOL)) %>% arrange(adj.P.Val)
  ...
}

```

Control을 기준으로 5개 시점(2/4/6/8/10주) 각각과 비교하는 선형모델. eBayes()는 표본이 적을 때(n=3) 통계적 안정성을 높여주는 limma의 핵심 스텝 (분산 추정치를 유전자 간에 shrinkage).

실행 결과

시점별 유의 유전자 수: 2주 186개, 4주 106개, 6주 644개, 8주 141개, 10주 216개 (같은 고정 기준, 완화 없이). 6주에서 유의 유전자가 급증하는 패턴이 나중에 TF activity에서도 똑같이 재현됨.

3.3 03_tf_activity.R — TF activity 추론 (마우스)

GSE222576_mouse_CCl4/03_tf_activity.R

GSE135251과 완전히 같은 shared 함수를 재사용하되, organism="mouse"만 다르게 지정.

```

common_probes = intersect(rownames(expr), probe_map$PROBEID)
expr_sub = expr[common_probes, ]
symbol_vec = probe_map$SYMBOL[match(common_probes, probe_map$PROBEID)]
expr_sym = avereps(expr_sub, ID = symbol_vec)

```

여러 probe가 같은 유전자를 가리키는 경우가 있어(예: 2개 probe가 둘 다 Prrx1), limma::avereps()로 같은 symbol의 probe를 평균내서 하나로 합침 — 표준적인 microarray 관행.

```
tf_result = run_tf_activity(expr_sym, organism = "mouse", cache_dir = "shared/cache")
```

이 한 줄이 이 프로젝트의 재사용 설계가 제대로 작동하는지 보여주는 지점입니다. RNA-seq이든 microarray든, 사람이든 마우스든, "gene symbol x sample" 행렬만 준비되면 완전히 같은 함수 호출로 TF activity가 나옵니다.

실행 결과 (+ 발견한 데이터 품질 문제)

670개 TF 평가, 292개 유의. 그런데 상위 리스트에 A0A0R4J082 같은 UniProt accession이 "TF"로 섞여 있는 걸 발견해서, tf_activity.R의 build_collectri_direct()에 정규식 필터를 추가하고 재실행 → 547개 TF로 정리됨. 최종 상위 TF: Prrx1(근섬유아세포 활성화의 대표적 마스터 TF), Scx, Hoxd3, Kdm5b, Tef, Klf2 — 특히 Prrx1, Twist1, Sox9, Notch1이 정확히 CCl4 6주차에서 피크를 찍는 패턴이 나와서, 이 논문(GSE222576 원 논문)이 설명한 "initiation-progression-tolerance" 3단계 구조와도 맞아떨어짐.

4. 데이터셋 3 — GSE74605 (마우스 TAA 시계열)

TAA(thioacetamide)로 간염유발을 유발한 마우스 모델 (Naive, 1/4/6주, n=3-4). 화학독성 축의 2번째 독립 데이터셋 — CCl4 하나의 결과가 우연이었는지 검증하기 위해, 사용자 요청으로 견고성(robustness) 검증 차원에서 추가.

4.1 01_download_qc.R — 비정규화 신호 다운로드와 quantile 정규화

GSE74605_mouse_TAA/01_download_qc.R

Illumina BeadChip 플랫폼이라 CCl4 데이터셋(Affymetrix)과 원자료 형식이 또 다름 — raw CEL 파일 대신 "non-normalized signal" 텍스트 파일 형태로 제공됨.

파일 구조 파악 과정

이 파일은 상단에 주석 3줄이 있고, 컬럼이 "샘플명, Detection Pval" 쌍이 반복되는 특이한 구조였습니다. 코드를 짜기 전에 파일을 직접 다운로드해서 처음 몇 줄을 확인하고 정확한 skip 줄 수와 컬럼 패턴을 파악한 뒤 파싱 코드를 작성했습니다.

```
raw = read.delim(nn_file, skip = 4, header = TRUE, check.names = FALSE)
signal_cols = colnames(raw)[colnames(raw) != "ID_REF" & colnames(raw) != "Detection Pval"]
signal = raw[, signal_cols]
rownames(signal) = raw$ID_REF
```

5번째 줄이 실제 헤더. "Detection Pval" 컬럼(신뢰도 지표)은 제외하고 실제 발현신호(signal) 컬럼만 추출.

```
group_levels = c("Naive", "TAA_1w", "TAA_4w", "TAA_6w")
sample_meta = data.frame(sample = signal_cols) %>%
  mutate(
    group = case_when(
      str_detect(sample, "^Naive") ~ "Naive",
      str_detect(sample, "^Week 1 ") ~ "TAA_1w",
      str_detect(sample, "^Week 4 ") ~ "TAA_4w",
      str_detect(sample, "^Week 6 ") ~ "TAA_6w"
    ), group = factor(group, levels = group_levels))
```

샘플명("Naive A", "Week 4 D" 등) 문자열에서 그룹을 파싱.

```
signal[signal <= 0] = NA
log_signal = log2(signal)
norm_expr = normalizeBetweenArrays(log_signal, method = "quantile")
```

log2 변환 후 limma::normalizeBetweenArrays()로 quantile 정규화. neqc()가 요구하는 negative control probe 정보가 이 처리된 파일엔 없어서, non-normalized Illumina 데이터에 대한 표준 대안인 quantile 정규화를 사용.

실행 결과

25,697 probes x 13 samples. 그룹: Naive 3, 1주 3, 4주 4, 6주 3.

4.2 02_deg_limma.R / 4.3 03_tf_activity.R

구조는 GSE222576과 거의 동일 (limma 선형모델 + eBayes, illuminaMousev2.db로 probe->symbol 매핑, avereps로 중복 probe 평균). 차이점은 annotation 패키지(illuminaMousev2.db)와 기준군 이름(Naive)뿐이라 코드 자체는 재사용에 가깝게 작성됨 — 아래는 두 스크립트에서 이 데이터셋에 특화된 부분만 발췌.

```
probe_map = AnnotationDbi::select(illuminaMousev2.db,  
                                   keys = rownames(expr), columns = "SYMBOL", keytype =  
                                   "PROBEID")
```

Illumina MouseRef-8 v2.0 (GPL6885) 전용 annotation 패키지 사용.

실행 결과

DEG: 1주 377개, 4주 425개, 6주 752개 (시간이 지날수록 늘어나는 합리적인 패턴). TF activity: 535개 평가, 256개 유의. 상위 TF 중 Smad4, Smad1, Smad5, Smad9(전부 TGF- β /SMAD 경로)가 Naive부터 6주까지 단조증가하는 클러스터를 이뤄서, 이 데이터셋 하나만으로도 TGF- β 경로가 핵심임을 보여주는 결과가 나왔습니다.

5. 교차검증 —

cross_dataset_metaanalysis/01_combine_tf_scores.R

3개 데이터셋 각각의 03_tf_activity.R로 만든 tf_stats.csv(TF별 p-value)와 tf_activity.rds(그룹별 평균 activity)를 읽어서 하나로 합치는 최종 단계. 이 프로젝트에서 가장 방법론적으로 신경 쓴 부분입니다.

5.1 세 데이터셋 결과 불러오고 방향 계산

```
load_dataset_tf = function(spec) {  
  stats = read_csv(file.path(spec$dir, "data", "tf_stats.csv"), show_col_types = FALSE) %>%  
    transmute(TF = toupper(TF), p_value, padj)  
  act = readRDS(file.path(spec$dir, "data", "tf_activity.rds"))  
  tf_group = act$tf_group  
  rownames(tf_group) = toupper(rownames(tf_group))  
  direction = sign(tf_group[, ncol(tf_group)] - tf_group[, 1])  
  stats %>% mutate(direction = direction[TF])  
}
```

사람 유전자 심볼은 대문자(TP53), 마우스는 첫글자만 대문자(Trp53)라 toupper()로 맞춰서 매칭. direction은 "가장 진행된 그룹의 평균 activity - 기준 그룹의 평균 activity"의 부호 — 단순 유의성뿐 아니라 실제로 증가/감소하는 방향까지 뒤에서 검증에 사용.

5.2 Fisher's method 결합 + 엄격한 선별 기준

왜 Fisher's method만으로는 부족한가

처음 2개 데이터셋(사람+CCI4)만으로 돌렸을 때, YAP1 같은 유전자가 사람에서는 극도로 유의(padj~1e-18)하지만 마우스에서는 전혀 유의하지 않은데도(padj=0.74) 결합 p-value는 유의하게 나오는 걸 발견했습니다. 표본 크기가 훨씬 큰 사람 데이터(n=216)가 결합 결과를 지배해버리는 Fisher's method의 잘 알려진 약점입니다. 그래서 "결합 p-value가 유의"보다 훨씬 엄격한 "3개 데이터셋 모두에서 각각 독립적으로 유의 + 방향까지 일치"를 1차 선별 기준으로 삼았습니다.

```
combined = merged %>%  
  rowwise() %>%  
  mutate(p_fisher = fisher_combine(c_across(all_of(p_cols)))) %>%  
  ungroup() %>%  
  mutate(  
    padj_fisher = p.adjust(p_fisher, method = "BH"),  
    all_significant = if_all(all_of(padj_cols), ~ . < 0.05),  
    direction_concordant = (rowSums(across(all_of(dir_cols)) == 1) == length(dir_cols)) |  
                           (rowSums(across(all_of(dir_cols)) == -1) == length(dir_cols)),  
    true_consensus = all_significant & direction_concordant  
  ) %>%  
  arrange(desc(true_consensus), padj_fisher)
```

all_significant: 3개 데이터셋 padj 전부 < 0.05. direction_concordant: 3개 데이터셋 방향이 전부 +1이거나 전부 -1(즉 다 같은 방향). 이 두 조건을 모두 만족하는 TF만 true_consensus = TRUE.

실행 결과

3개 데이터셋 공통 평가 TF 511개. Fisher 결합만으로는 460개가 유의하지만, "3개 모두 유의" 기준으로는 105개, 여기에 방향 일치까지 더하면 최종 56개로 좁혀짐. 상위 15개: SMARCA4, NR2E1, SMAD6, MEOX1, HMGB2, SRSF2, HOXD3, THAP11, SMAD7, FOXJ1, KLF2, LEF1, E2F7, MEOX2, SMAD3. SMAD3(촉진)과 SMAD6/SMAD7(억제) 세 개가 동시에 나온 게 특히 인상적이었습니다 — "섬유화가 진행될수록 TGF- β 신호를 누르는 브레이크가 풀린다"는 일관된 메커니즘 스토리가 3개 데이터셋 전부에서 재현된 것입니다.

5.3 결과 시각화

```
zscore_rows = function(mat) {  
  m = rowMeans(mat, na.rm = TRUE)  
  s = apply(mat, 1, sd, na.rm = TRUE)  
  s[s == 0] = 1  
  sweep(sweep(mat, 1, m, "-"), 1, s, "/")  
}  
z_blocks = map2(tf_groups, names(datasets), function(m, nm) {  
  z = zscore_rows(m[avail_tf, , drop = FALSE])  
  colnames(z) = paste0(nm, "_", colnames(m))  
  z  
})  
combined_mat = reduce(z_blocks, cbind)
```

세 데이터셋의 TF activity 절대 스케일이 서로 다르기 때문에(플랫폼이 다르므로), heatmap 전체를 한 번에 z-score 처리하면 스케일이 큰 데이터셋(주로 마우스)이 그림을 지배해버립니다. 그래서 각 데이터셋 안에서 따로따로 z-score를 계산한 뒤에 옆으로 붙이는 방식을 씀 — 세 데이터셋 모두 "자기 안에서의 상대적 패턴"이 공평하게 보이도록.

6. 코드 이후: 문헌 검증과 실험 설계 (요약)

여기서부터는 R 코드가 아니라 문헌 조사·수작업 정리 산출물입니다. 전체 내용은 저장소의 `candidate_shortlist.csv`와 `ARID1A_experimental_plan.md`에 있고, 여기서는 흐름만 요약합니다.

- 56개 consensus TF를 druggability와 기존 간 문헌 여부로 triage.
- 처음에는 MEOX1, NR2C2, ARID1A, NUPR1을 "완전히 새로운 후보"로 판단했으나, 검색어를 바꿔가며 (별칭, 기전 용어, 최근 연도) 재검증한 결과 4개 중 4개 모두 이미 관련 논문이 있다는 걸 확인 — MEOX1/NR2C2는 이미 간섬유화 직접 논문 있음, ARID1A/NUPR1은 간세포·담관세포에서 보호적 역할로 이미 연구됨(HSC 자체에서는 미검증).
- 재검증을 두 번 거쳐도 문헌이 전혀 없는 건 THAP11뿐 — 가장 확실한 "신규" 후보지만 알려진 억제제가 없음.
- 최종적으로 ARID1A(SMARCA4와 같은 SWI/SNF 복합체라는 기전적 근거)와 THAP11(가장 깨끗한 문헌 공백) 둘을 Phase 1 세포주 실험에서 병행 검증하는 계획을 세움 — LX-2(간성상세포) vs AML12(간세포) 대조 실험을 최우선으로 두어, bulk 데이터의 "증가" 패턴이 세포구성 혼입 때문인지 진짜 세포종류-특이적 기능 때문인지부터 가리는 구조.
- 8주 일정, 약 860만-1,450만원(2타겟 병행 기준) 예산으로 Phase 1 계획서를 작성.

깃허브 저장소

전체 코드와 결과는 github.com/bioinform25/pharmacology2254 (main 브랜치)에 커밋되어 있습니다. PR #2에 이번 프로젝트의 전체 리뷰/정정 과정이 코멘트로 남아 있습니다.